


Lecture 3: Deep Learning "Hello World": MNIST
Backpropagation by Hand
Regression versus Classification

# Deep Learning in Classical and Quantum Physics

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

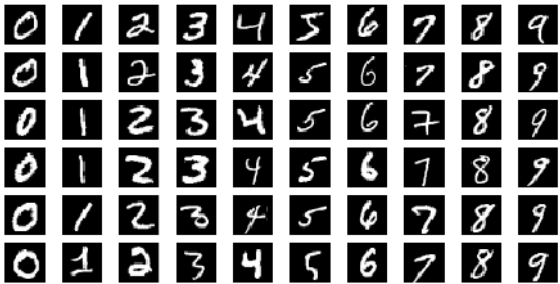
# 1

# Deep Learning “Hello World”: MNIST



JAGIELLONIAN UNIVERSITY
IN KRAKOW

# Example #1: *Hello world!* MNIST



- hand-written digits $0, \ldots, 9$
- $28 \times 28$ grey-scale pixels, values $0 \ldots 255$
- 60 000 training and 10 000 test images
- classes nearly balanced: guessing gives $\approx 10\%$

## Splitting the data

**training** — fit the parameters
**validation** — choose settings
**test** — used *once*, for the final quality

LeCun et al. (1998)

# Digit Classification as Non-Linear Function Fitting

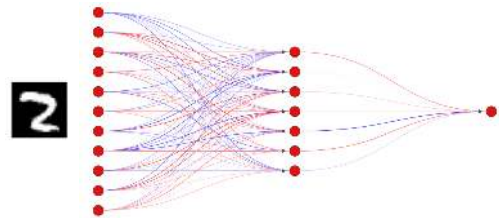
Somewhere there is an unknown function

$$G : \vec{x} \in \mathbb{R}^{784} \to y \in \{0, \ldots, 9\}$$

from pixels to digits — the high-dimensional cousin of $g(x)$ from the universal approximation slide. We only see finitely many samples of it.

**The naive model:** treat it as **regression**

- *one* output node, $\bar{y} = \sigma(\cdot) \in (0, 1)$
- target: the digit divided by 10
- loss: mean squared error
- prediction: $10\,\bar{y}$, rounded



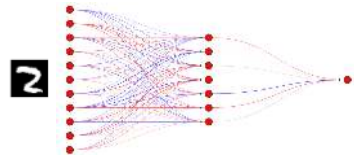
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$784 \to 21$ sigmoid nodes $\to 1$ sigmoid output

# Feed-Forward Pass

under the hood

A mini-batch of $N_{\mathcal{D}}$ images, each flattened to $m = 784$ pixels: $X \in \mathbb{R}^{N_{\mathcal{D}} \times m}$



| step | operation | shape |
|---|---|---|
| 1. first linear layer | $\vec{h} = X\vec{W}^{(1)} + \vec{b}^{(1)}$ | $[N_{\mathcal{D}}, m] \times [m, N^{(1)}] = [N_{\mathcal{D}}, N^{(1)}]$ |
| 2. activation | $\bar{\vec{h}} = \sigma(\vec{h})$ | $[N_{\mathcal{D}}, N^{(1)}]$ |
| 3. second linear layer | $\vec{g} = \bar{\vec{h}}\,\vec{W}^{(2)} + b^{(2)}$ | $[N_{\mathcal{D}}, N^{(1)}] \times [N^{(1)}, 1] = [N_{\mathcal{D}}, 1]$ |
| 4. output activation | $\bar{\vec{y}} = \sigma(\vec{g})$ | $[N_{\mathcal{D}}, 1]$ |

Loss on the mini-batch: $\mathcal{L} = \dfrac{1}{2N_{\mathcal{D}}} \left\| \bar{\vec{y}} - \vec{y} \right\|^2$ ($\vec{y}$: true digits/10)

# Parameter Update by Hand: the Output Layer

under the hood

Forward pass on a mini-batch (rows = images): $H = XW^{(1)} + \mathbf{b}^{(1)}$, $\bar{H} = \sigma(H)$, $G = \bar{H}W^{(2)} + b^{(2)}$, $\bar{Y} = \sigma(G)$, $\mathcal{L} = \frac{1}{2N_{\mathcal{D}}} \sum_n (\bar{y}_n - y_n)^2$

One weight of the output layer, $W_j^{(2)}$; the chain rule follows the forward pass backwards:

$$\frac{\partial \mathcal{L}}{\partial W_j^{(2)}} = \frac{1}{N_{\mathcal{D}}} \sum_n \underbrace{(\bar{y}_n - y_n)}_{\partial \mathcal{L}/\partial \bar{y}_n} \underbrace{\bar{y}_n(1-\bar{y}_n)}_{\partial \bar{y}_n/\partial g_n} \underbrace{\bar{h}_{nj}}_{\partial g_n/\partial W_j^{(2)}}, \qquad \sigma'(z) = \sigma(z)\,(1-\sigma(z))$$

Collect the error signal of the output, one number per image:

$$\boldsymbol{\delta}^{(2)} = (\bar{Y} - Y) \odot \bar{Y} \odot (1 - \bar{Y}) \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial W^{(2)}} = \frac{1}{N_{\mathcal{D}}} \bar{H}^T \boldsymbol{\delta}^{(2)}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = \frac{1}{N_{\mathcal{D}}} \sum_n \delta_n^{(2)}$$

shapes: $\bar{H}^T \boldsymbol{\delta}^{(2)}$: $[N^{(1)}, N_{\mathcal{D}}] \times [N_{\mathcal{D}}, 1] = [N^{(1)}, 1]$, the shape of $W^{(2)}$; $\odot$: element-wise product

# Parameter Update by Hand: the Hidden Layer

under the hood

A hidden weight $W^{(1)}_{ij}$ acts on the loss through neuron $j$, which feeds the output:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}_{ij}} = \frac{1}{N_{\mathcal{D}}} \sum_n \underbrace{\delta^{(2)}_n \, W^{(2)}_j}_{\partial \mathcal{L}/\partial \bar{h}_{nj}} \underbrace{\bar{h}_{nj}(1-\bar{h}_{nj})}_{\partial \bar{h}_{nj}/\partial h_{nj}} \underbrace{x_{ni}}_{\partial h_{nj}/\partial W^{(1)}_{ij}}$$

The error signal is **propagated back** through $W^{(2)}$:

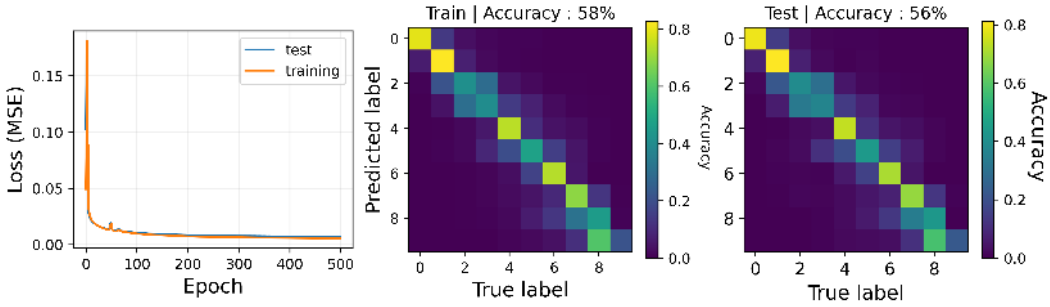
$$\boldsymbol{\delta}^{(1)} = \left(\boldsymbol{\delta}^{(2)} \, W^{(2)\,T}\right) \odot \bar{H} \odot (1-\bar{H}) \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{1}{N_{\mathcal{D}}} X^T \boldsymbol{\delta}^{(1)}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \frac{1}{N_{\mathcal{D}}} \sum_n \boldsymbol{\delta}^{(1)}_n$$

shapes: $\boldsymbol{\delta}^{(2)} W^{(2)\,T}$: $[N_{\mathcal{D}}, 1] \times [1, N^{(1)}]$; $X^T \boldsymbol{\delta}^{(1)}$: $[m, N_{\mathcal{D}}] \times [N_{\mathcal{D}}, N^{(1)}] = [m, N^{(1)}]$

Gradient-descent update, for every parameter: $\theta \leftarrow \theta - \eta \, \partial \mathcal{L}/\partial \theta$

**The pattern is general (backpropagation):** $\boldsymbol{\delta}^{(l)} = \left(\boldsymbol{\delta}^{(l+1)} W^{(l+1)\,T}\right) \odot \sigma'\left(H^{(l)}\right)$, $\partial \mathcal{L}/\partial W^{(l)} = \frac{1}{N_{\mathcal{D}}} \bar{H}^{(l-1)\,T} \boldsymbol{\delta}^{(l)}$. These formulas agree with automatic differentiation (JAX) to machine precision; in practice PyTorch or JAX do this bookkeeping for us.

# Single Output and MSE: the Result

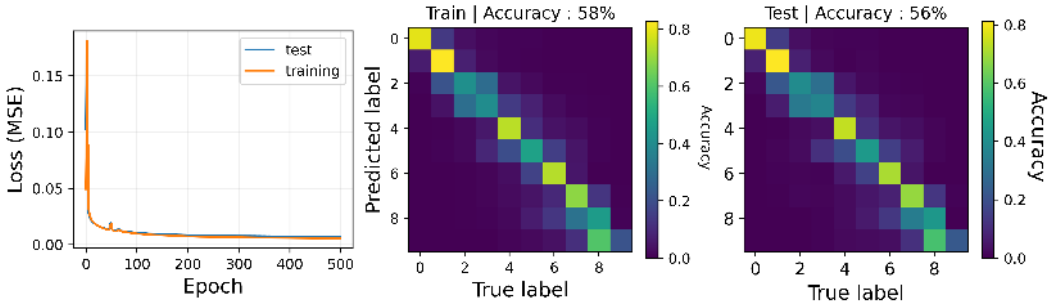



**Confusion matrix:** row = predicted, column = true digit; accuracy = mean of the diagonal.

Single output node, MSE: about 58% training, 56% test accuracy.

*Better than guessing (10%) — but why so poor?*

# Single Output and MSE: the Result




**Confusion matrix:** row = predicted, column = true digit; accuracy = mean of the diagonal.

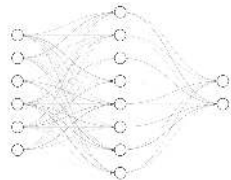
Single output node, MSE: about 58% training, 56% test accuracy.

*Better than guessing (10%) — but why so poor?*

One number on a line imposes an *ordering*: "3" lies between "2" and "4", and confusing 1 with 9 costs more than 1 with 2. Digits are *categories*, not quantities.

# Loss Function: Categorical Cross-Entropy





*ten* output nodes $z_0, \ldots, z_9$

$p = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$ (one-hot label)

$q = [0.1, 0, 0.7, 0.01, 0.1, 0.02, 0, 0.02, 0, 0.05]$

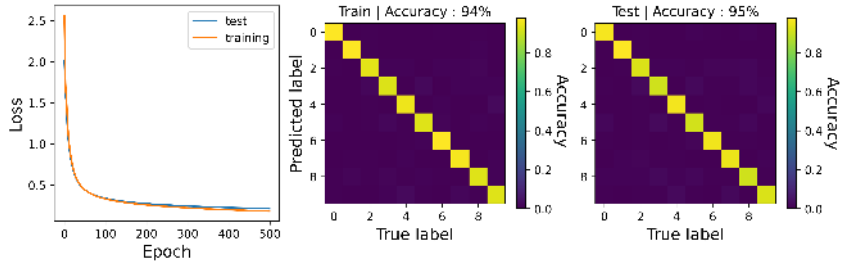
Normalisation (**softmax**): $q_k = q(k \mid x) = \dfrac{e^{z_k}}{\sum_{j=0}^{9} e^{z_j}}$

From the KL divergence (previous section):

$$\mathcal{L}_{\mathrm{CE}}(p, q) = -\sum_{k=0}^{9} p_k \log q_k = -\log q_{\text{true digit}}$$

- no ordering between classes
- network outputs a *probability* for each digit
- untrained network: $\mathcal{L}_{\mathrm{CE}} \approx \ln 10 \approx 2.3$

# Categorical Cross-Entropy: Result




Same hidden layer (21 sigmoid nodes), ten outputs, cross-entropy: about 94% training, 95% test accuracy.

| Lab 009 (Adam, mini-batches of 64, 20 epochs) | test accuracy |
|---|---|
| 1 output, MSE | 65.2% |
| 10 outputs, MSE on one-hot labels | 93.8% |
| 10 outputs, cross-entropy | 94.8% |

**Lesson:** the *representation of the output* and the *loss* matter as much as the network.

# Summary

1. **MNIST**: digit classification as non-linear function fitting
2. **Backpropagation**: the chain rule applied from the loss backwards; the error signal $\delta^{(l)} = (\delta^{(l+1)} W^{(l+1)T}) \odot \sigma'(H^{(l)})$
3. **One output with the mean squared error** imposes an ordering on the digits
4. **Ten outputs with the categorical cross-entropy** treat them as categories, and the network outputs a probability for each digit
5. **Lesson**: the representation of the output and the loss matter as much as the network

# Next Lecture

1. CIFAR-10: images with spatial structure, and convolutional neural networks
2. Regularisation, class activation maps and adversarial examples
3. A physics example: the 2D Ising model and its critical temperature

# Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar

